SYSTEM DESIGN & ARCHITECTURE

Architecture Explanation (3-Tier System Design)

This system follows a **3-tier architecture**, which separates the application into three independent layers: Presentation, Logic, and Data. This separation improves scalability, maintainability, and security.

---

1. Presentation Tier (Frontend)

The **Presentation Layer** consists of a single-page application built using **React.js or Vue.js**.

Responsibilities:

- Provides the user interface for the system
- Handles user interactions (login, task creation, file upload, etc.)
- Sends and receives data from backend via REST APIs
- Displays real-time updates to the user

Key Component:

- **UI (React/Vue SPA)**: Acts as the client-side application that communicates with the backend

2. Logic Tier (Backend)

The **Logic Layer** is implemented using a backend framework such as **Node.js or Django**. It contains the core business logic of the system.

Responsibilities:

- Processes all client requests
- Handles authentication and authorization using JWT
- Manages task operations (create, update, delete, assign)
- Coordinates file handling and storage operations
- Acts as an intermediary between frontend and database

Key Components:

- **API Gateway**
  - Entry point for all frontend requests
  - Routes requests to appropriate services
- **Auth Controller**
  - Handles login/register logic
  - Verifies JWT tokens for secure access
- **Task Service**
  - Implements business logic for task management
  - Communicates with database for CRUD operations

- **File Middleware (Multer)**
    - Processes file uploads before storing them
    - Validates and prepares files for storage

3. Data Tier (Database & Storage)

The **Data Layer** is responsible for persistent data storage and file management.

Responsibilities:

- Stores application data (users, tasks, teams)
- Manages file storage (documents, images, attachments)
- Ensures data integrity and persistence

Key Components:

- **Database (MySQL / MongoDB)**
    - Stores structured or semi-structured data
    - Handles queries from backend services
- **Storage (Cloudinary / Local File System)**
    - Stores uploaded files
    - Returns file URLs for frontend access

Data Flow Explanation

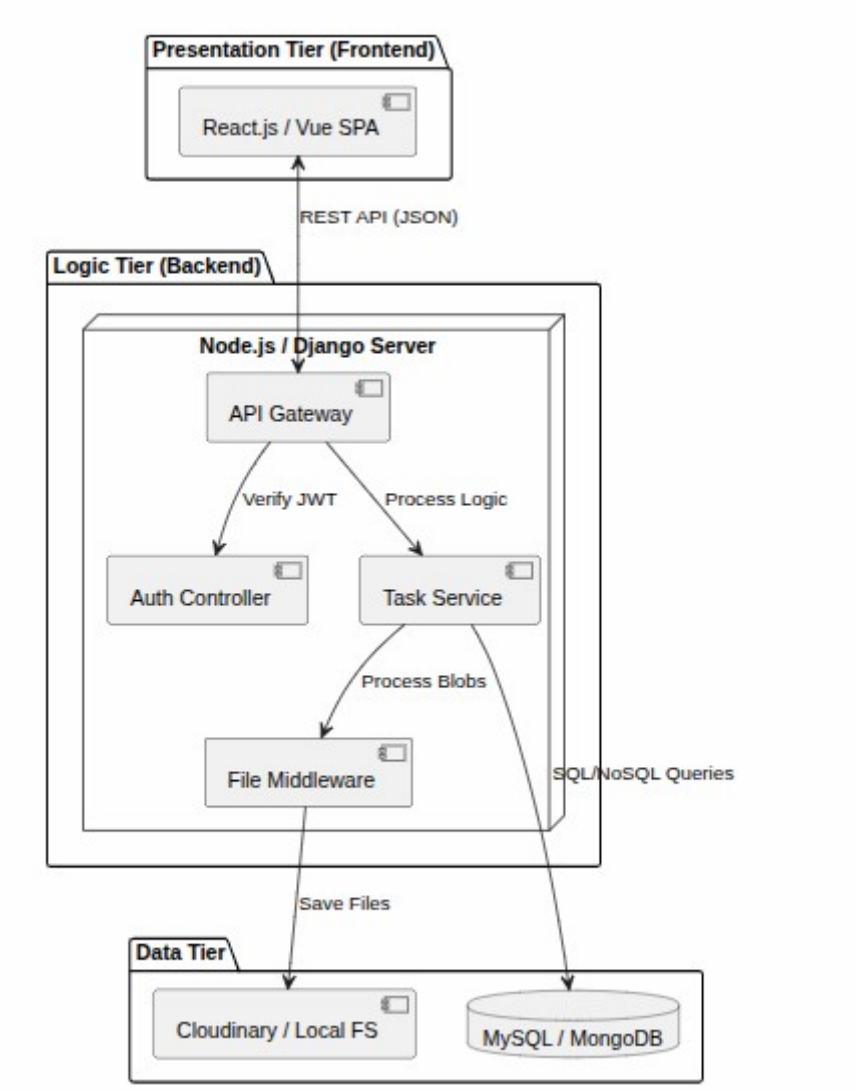
1. User interacts with the **Frontend (UI)**
2. UI sends a **REST API request (JSON)** to the backend
3. Request reaches the **API Gateway**
4. Gateway routes request to:
    - Auth Controller (for login/verification)
    - Task Service (for task operations)
5. Task Service interacts with:
    - Database for data storage/retrieval
    - File middleware for uploads
6. Files are stored in **Cloud storage or local filesystem**
7. Response is sent back to frontend and displayed to user

Why This Architecture is Effective

- **Separation of concerns** → each layer has a clear responsibility
- **Scalability** → backend and frontend can scale independently
- **Maintainability** → easier debugging and updates
- **Flexibility** → supports different databases and storage systems

Security Considerations

- JWT-based authentication ensures secure access control
- API Gateway acts as a security checkpoint
- Backend validates all requests before processing
- File uploads are handled securely via middleware
- Database access is restricted to backend services only

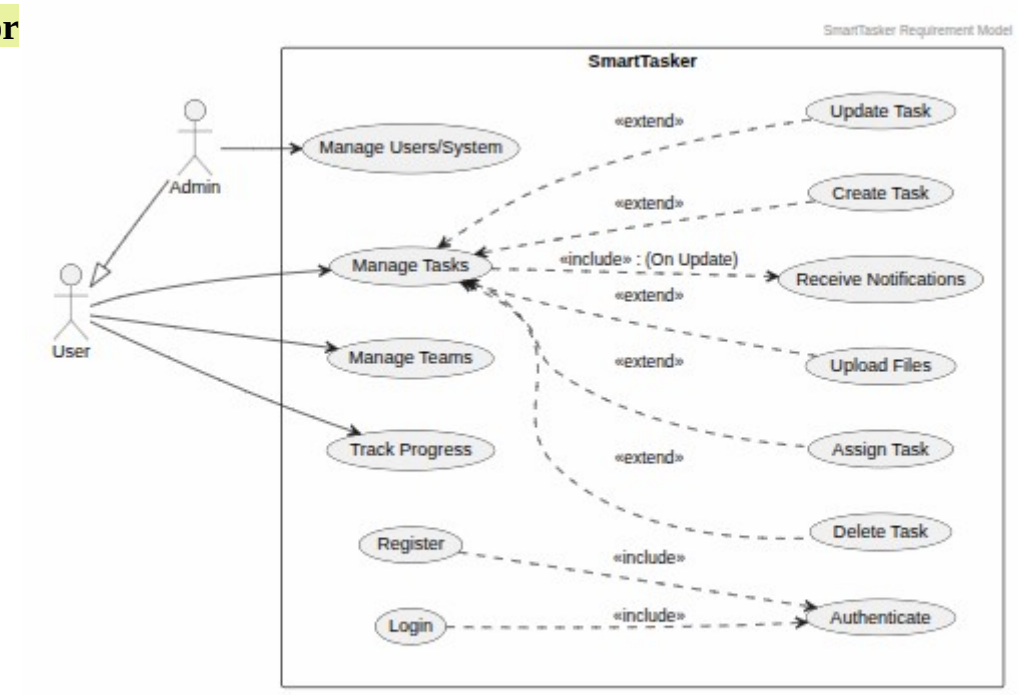


UML DIAGRAMS

1. Use Case Diagram

The Use Case diagram defines the **functional scope** and **user boundaries** of SmartTasker.

1. Actor



Hierarchy (Inheritance)

The diagram employs **Generalization** Admin → User. The **Admin** inherits all functions of the **User** (Task CRUD, Team collaboration) while maintaining exclusive access to **System/User Management**. This reduces redundancy and reflects the system's **Role-Based Access Control (RBAC)**.

2. Authentication Module (Include)

The **Authenticate (UC3)** use case is an <<include>> for both **Login** and **Register**. This signifies that identity verification is a mandatory, non-optional step within the authentication flow, ensuring system security.

3. Task Management Framework (Extend)

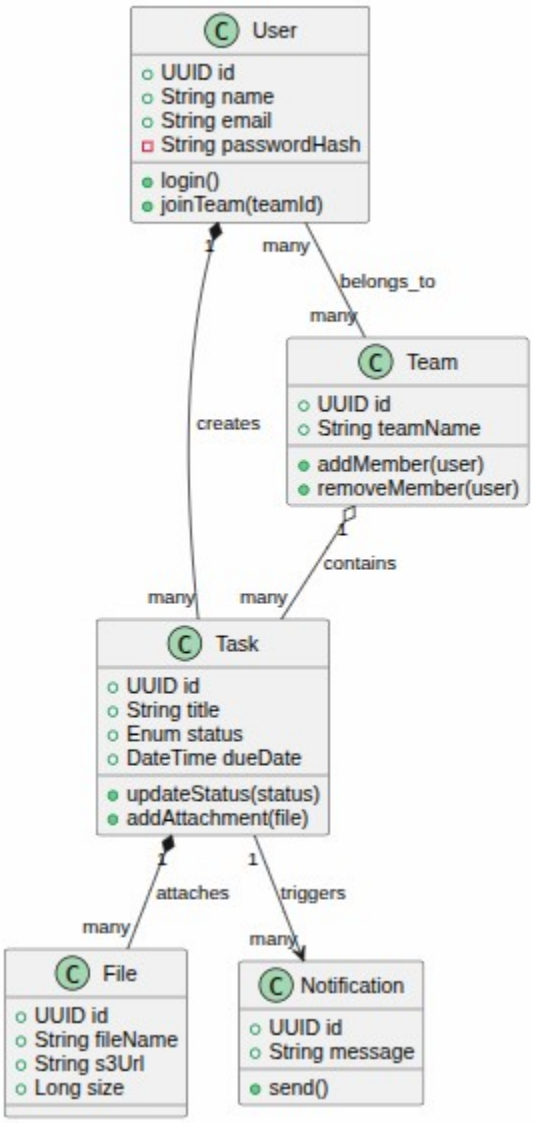
Manage Tasks (UC4) serves as the primary gateway. Specific actions—**Create**, **Update**, **Delete**, **Assign**, and **Upload**—are modeled as <<extend>>. This indicates that these features are optional branches available to the user only after they have accessed the task management module.

4. Event-Based Notifications

The system triggers **Receive Notifications (UC11)** as an <<include>> during task modifications. This demonstrates an automated business rule where the system ensures stakeholders stay informed of progress changes.

2. Class Diagram :

The Class Diagram serves as the **Blueprint for the Data Layer**, defining how the system stores information and how objects interact within the backend.



1. Encapsulation & Security Logic

- **Private Data Masking (Red Icon):** The `passwordHash` is encapsulated as **Private**. Technically, this ensures the system follows the **Principle of Least Privilege**, preventing sensitive credentials from being exposed during object serialization or API responses.
- **Public Interface (Green Icon):** Methods like `updateStatus()` and `login()` define the **Public API** of the class. This forces all state changes through validated logic, ensuring a task cannot reach an invalid state (e.g., jumping from "Created" to "Archived" without review).

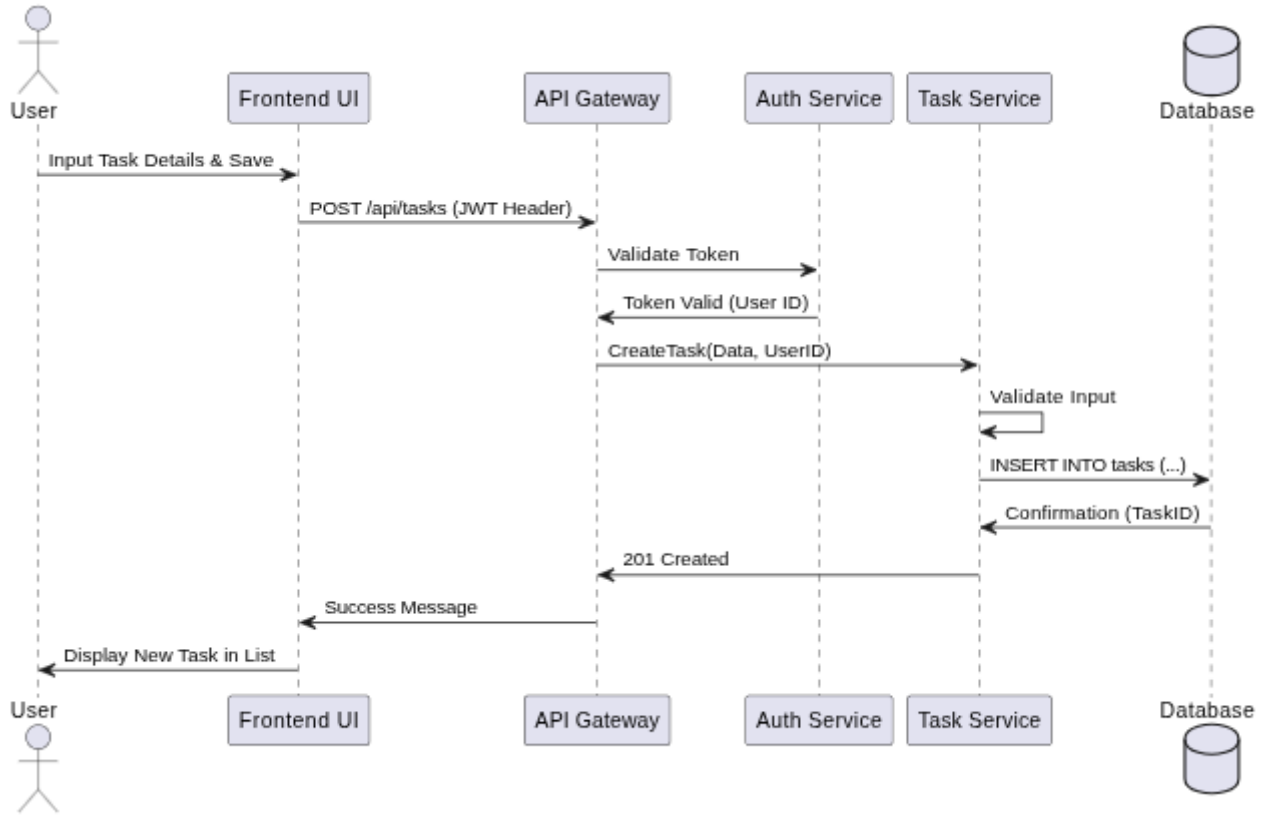
2. Relationship & Lifecycle Logic

- **Strong Ownership (Composition ◆):** Used for **Task ➡ File**. In a Task Manager, if a task is deleted, its attachments are no longer relevant. This relationship ensures **Cascading Deletes** in the database, preventing storage bloat from "orphan" files.
- **Weak Association (Aggregation ◇):** Used for **Team ➡ Task**. This allows for **Resource Reassignment**. If a team is deleted, the tasks can remain in the system for archival or be moved to a different team, preserving historical work data.

- **Many-to-Many Association:** Between **User ➡ Team**. This supports the collaborative requirement where users participate in multiple projects and teams consist of various members.

3. Sequence Diagram

This sequence diagram illustrates the **end-to-end process of creating a task** in the SmartTasker system. It shows how different system components interact in a step-by-step manner, starting from user input to final task confirmation.



1. User Interaction

The process begins when the **User enters task details** in the frontend interface.
The frontend collects the input and prepares it for submission.

- The **Frontend sends a POST request (`/tasks`)** to the backend
- The request includes a **JWT token** for authentication

2. Backend Request Handling

Once the request reaches the backend:

- The **Backend forwards the token to AuthService**
- AuthService validates whether the user is authenticated

Authentication Check

Case 1: Invalid Token

- AuthService rejects the token
- Backend returns a **401 Unauthorized error**
- Frontend displays an error message to the user

Case 2: Valid Token

- AuthService confirms authentication
- Backend proceeds to business logic processing

3. Task Processing (Business Logic Layer)

After successful authentication:

- Backend sends the request to **TaskService**
- TaskService validates the input data

Case1: Validation Failure

- TaskService detects invalid input
- Returns error to backend
- Backend sends **validation error** to frontend
- User is prompted to correct input

Case2: Validation Success

- TaskService processes the request successfully

4. Database Operation

If validation is successful:

- TaskService sends a request to the **Database**
- A new task record is inserted
- Database confirms successful storage

5. Notification System

After task creation:

- TaskService triggers the **NotificationService**
- Notification system sends alerts to relevant team members
- Confirmation is returned after successful delivery

6. Response to User

Finally:

- TaskService confirms task creation to backend
- Backend sends a success response to frontend
- Frontend displays a **success message to the user**

4. Activity Diagram

This activity diagram represents the **complete lifecycle of a task** in the SmartTasker system, from creation to completion. It illustrates workflow steps, decision points, parallel processes, and iterative updates.

1. Task Creation Phase

The process begins when a **User creates a task**.

- The task is then **assigned to a team member**
- The system checks whether the task has been properly assigned

Decision: Team Assigned?

- **Yes** → Workflow continues normally
- **No** → Task is **reassigned** until a valid assignment is made

2. Parallel Execution Phase

Once a task is successfully assigned, two processes occur **in parallel**:

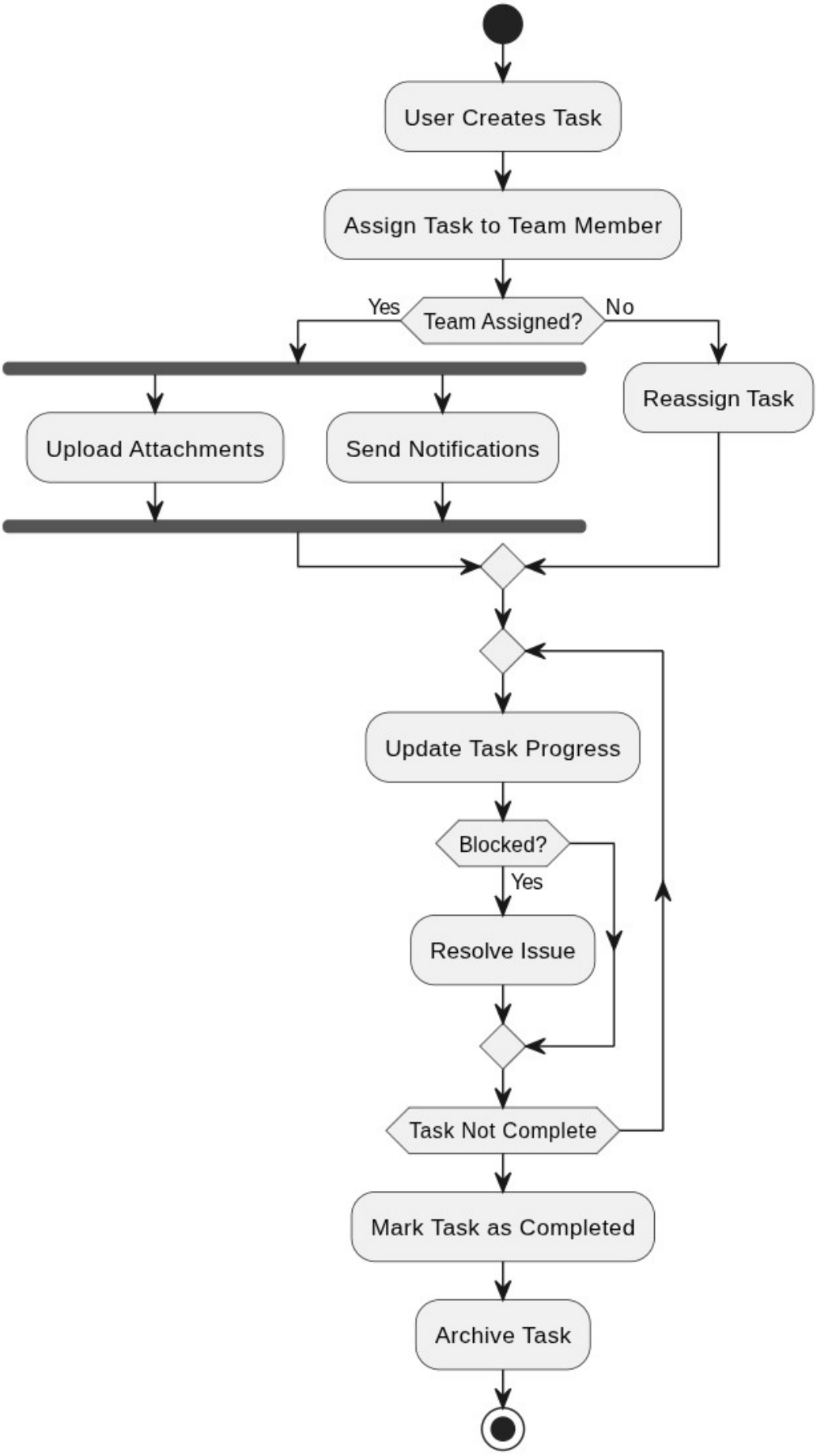
A. Upload Attachments

- Users can upload supporting files (documents, images, etc.)
- These files are linked to the task

B. Send Notifications

- The system notifies relevant team members
- Ensures collaboration and awareness

These two activities run simultaneously to improve efficiency.


User Creates Task
Assign Task to Team Member
Team Assigned?
Yes
No
Reassign Task
Upload Attachments
Send Notifications
Update Task Progress
Blocked?
Yes
Resolve Issue
Task Not Complete
Mark Task as Completed
Archive Task

3. Task Progress Update Phase

After initial setup:

- The task enters the **progress tracking stage**
- Users continuously **update task status**

Decision: Blocked?

- **Yes** →
  - The system triggers **issue resolution**
  - Problems are addressed before continuing
- **No** →
  - Task continues normal progress flow

4. Completion Check Loop

The system repeatedly checks:

- Whether the task is completed or still in progress

Decision: Task Not Complete?

- If **not complete** → return to progress update loop
- If **complete** → proceed to final steps

5. Task Completion Phase

Once all work is finished:

- Task is **marked as completed**
- System updates final status

6. Archiving Phase

- Completed tasks are moved to **archive storage**
- This ensures:
  - Clean active workspace
  - Historical record maintenance

7. End of Process

The workflow terminates after archiving the task.